LP-III · FINAL YEAR B.E. · PRACTICALS 1 – 5

Design and Analysis of Algorithms

Every practical explained from the ground up, with worked examples and probable viva / oral questions with answers

+	**Algorithm analysis basics**	Before you start
1	**Fibonacci numbers — recursive and non-recursive**	Practical 1
2	**Huffman encoding — Greedy strategy**	Practical 2
3	**0/1 Knapsack — Dynamic Programming**	Practical 3
4	**N-Queens with the first queen placed — Backtracking**	Practical 4
5	**Quick Sort — deterministic and randomized**	Practical 5
+	**Quick revision and cross-practical questions**	Revision

Based on the code in **github.com/qwerty7733/lp3**.

How to use: read the theory once, trace the worked example by hand, then cover the answers and test yourself on the questions. Answers are written the way you would say them in a viva — short first line, then the detail if the examiner digs deeper.

BEFORE YOU START

Algorithm analysis basics

Every DAA viva starts with a few warm-up questions before the examiner opens your file. These ideas are used in all five practicals, so learn them first.

What is an algorithm?

An **algorithm** is a finite sequence of well-defined steps that takes some input and produces the required output. A good algorithm has five properties: **input**, **output**, **definiteness** (each step is clear), **finiteness** (it stops) and **effectiveness** (each step can actually be carried out).

Measuring an algorithm

We compare algorithms by how their running time and memory grow as the input size *n* grows, not by seconds on one machine.

- **Time complexity** — number of basic operations (comparisons, additions) as a function of *n*.
- **Space complexity** — extra memory used: variables, tables, and the **recursion stack**.
- **Best / average / worst case** — the same algorithm can behave differently on different inputs of the same size. Quick Sort is the classic example (Practical 5).

Asymptotic notations

Notation	Meaning	Read as	Example
O(g(n)) — Big-O	upper bound: f(n) ≤ c·g(n) for large n	"grows no faster than"	Quick Sort worst case $O(n^2)$
Ω(g(n)) — Omega	lower bound: f(n) ≥ c·g(n)	"grows at least as fast as"	any comparison sort is Ω(n log n)
Θ(g(n)) — Theta	tight bound: both O and Ω	"grows exactly like"	iterative Fibonacci is Θ(n)

Common growth rates, slowest to fastest: **$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$**. The five practicals together touch almost all of these: O(1) space Fibonacci, O(n log n) Huffman and Quick Sort, O(n·W) knapsack, $O(2^n)$ recursive Fibonacci and O(N!) N-Queens.

Algorithm design strategies used in this lab

Strategy	Core idea	Practical
Recursion / Iteration	solve a problem in terms of a smaller copy of itself, or with a loop	1 – Fibonacci
Greedy	at each step take the choice that looks best right now; never undo it	2 – Huffman
Dynamic Programming	split into overlapping sub-problems, solve each once, store the answer in a table	3 – 0/1 Knapsack (and memoized Fibonacci)
Backtracking	build a solution step by step; the moment a partial solution breaks a rule, undo the last step and try the next option	4 – N-Queens
Divide and Conquer	split into independent sub-problems, solve recursively, combine	5 – Quick Sort
Randomized	use random choices inside the algorithm to avoid bad inputs	5 – Randomized Quick Sort

Recurrence relations and the Master theorem

A recursive algorithm's running time is written as a **recurrence**. For divide and conquer of the form **T(n) = a·T(n/b) + f(n)**, the **Master theorem** gives the answer by comparing f(n) with n^(log_b a):

- f(n) smaller → T(n) = Θ(n^(log_b a))
- f(n) equal → T(n) = Θ(n^(log_b a) · log n) ← Quick Sort / Merge Sort best case: T(n) = 2T(n/2) + n = Θ(n log n)
- f(n) larger → T(n) = Θ(f(n))

Recurrences like **T(n) = T(n−1) + n** (Quick Sort worst case) or **T(n) = T(n−1) + T(n−2) + 1** (recursive Fibonacci) are not in Master-theorem form; solve them by expanding (substitution / recursion tree).

Warm-up questions

Q1. What is the difference between an algorithm and a program?

An algorithm is the language-independent idea (steps); a program is that algorithm written in a particular programming language that a computer can run. One algorithm can have many programs.

Q2. Why do we use asymptotic analysis instead of measuring time in seconds?

Seconds depend on the machine, language, compiler and current load. Asymptotic analysis counts basic operations as a function of n, so it compares algorithms independent of hardware and tells us how they scale.

Q3. What is the difference between O, Ω and Θ?

O is an upper bound (worst it can grow), Ω is a lower bound, and Θ is a tight bound when both match. Saying "Quick Sort is $O(n^2)$" is true but loose for the average case; "average case Θ(n log n)" is precise.

Q4. What is space complexity? Does recursion use extra space?

Space complexity is the extra memory used as a function of n. Yes — every pending recursive call keeps a stack frame, so recursion depth d costs O(d) space even if the function itself uses no arrays.

Q5. What is the difference between Greedy and Dynamic Programming?

Greedy makes one locally best choice per step and never revisits it — fast, but correct only when the problem has the greedy-choice property (Huffman, fractional knapsack). DP considers all choices for every sub-problem and stores results — slower but always optimal when the problem has optimal substructure and overlapping sub-problems (0/1 knapsack).

Q6. What is the difference between Divide & Conquer and Dynamic Programming?

Both split a problem into sub-problems. In D&C the sub-problems are **independent** (Quick Sort's left and right halves never share work). In DP the sub-problems **overlap**, so storing their answers avoids recomputation (Fibonacci, knapsack).

Q7. What are P, NP and NP-complete problems?

P: solvable in polynomial time. NP: a given answer can be **verified** in polynomial time. NP-complete: the hardest problems in NP — if any one is solved in polynomial time, all of NP is. The 0/1 knapsack decision problem is NP-complete; the O(n·W) DP is only pseudo-polynomial.

PRACTICAL 1

Fibonacci numbers — recursive and non-recursive

Aim. Write recursive and non-recursive programs to calculate Fibonacci numbers and analyse their time and space complexity.

Code: `DAA_Fibonacci_pract1.py` — three functions: `fib_iterative`, `fib_recursive` (with a global call counter) and the bonus `fib_memo`.

The Fibonacci series

Each number is the sum of the two before it: **0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...**

RECURRENCE RELATION

$F(n) = F(n-1) + F(n-2)$ for $n \geq 2$, with **base cases** $F(0) = 0$ and $F(1) = 1$.

The ratio $F(n+1)/F(n)$ approaches the **golden ratio $\varphi \approx 1.618$**, which is why Fibonacci numbers grow exponentially and why the naive recursion takes about φ^n calls.

Method 1 — Iterative (non-recursive)

Keep only the last two numbers and slide forward n times:

```
a, b = 0, 1            # F(0), F(1)
repeat n times:
    a, b = b, a + b    # Python evaluates the right side first, so no temp variable is needed
return a               # a now holds F(n)
```

Trace for n = 5:

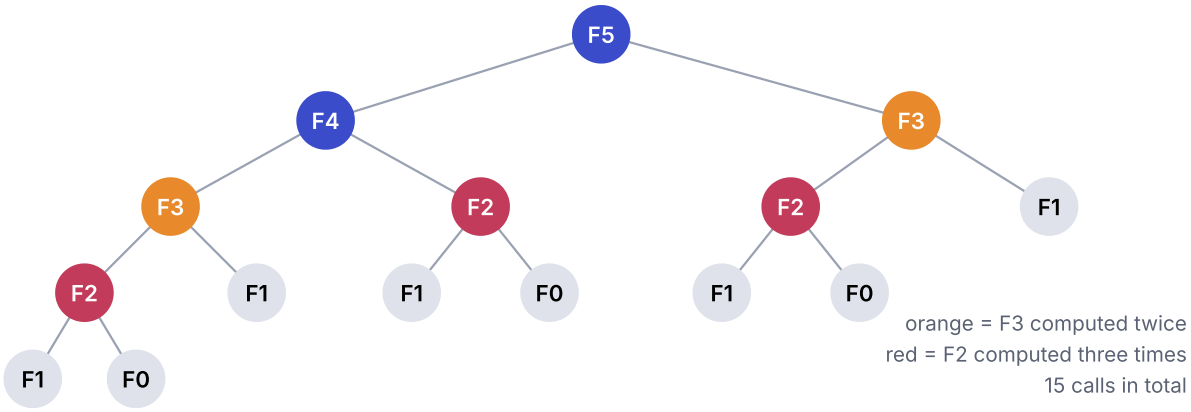
Step	a	b
start	0	1
1	1	1
2	1	2
3	2	3
4	3	5
5	**5**	8

Time O(n) — one loop of n iterations. **Space O(1)** — only two variables, whatever n is. (If you store the whole series in a list, space becomes O(n).)

Method 2 — Recursive

Translate the recurrence directly: `if n <= 1: return n` else `return fib(n-1) + fib(n-2)`.

The problem is **overlapping sub-problems**: the same values are recomputed again and again. The recursion tree for F(5) has 15 calls; F(3) is computed twice, F(2) three times.



Recursion tree of fib_recursive(5). Repeated colours are wasted, repeated work.

Number of calls. calls(n) = 1 + calls(n−1) + calls(n−2), which works out to **2·F(n+1) − 1**. So F(5) → 15 calls, F(10) → 177, F(20) → 21,891 and F(30) → **2,692,537** calls (exactly what the program prints).

Time O(2ⁿ) (tight bound Θ(φⁿ) ≈ Θ(1.618ⁿ)). **Space O(n)** — the tree is wide but only one root-to-leaf path is on the call stack at any moment, and its length is n.

Method 3 — Memoization (top-down DP)

Store each answer the first time it is computed (`memo[n] = ...`) and return the stored value next time. Each F(i) is now computed once → **Time O(n), Space O(n)** (dictionary + stack). The bottom-up version, filling an array from F(0) upward, is called **tabulation**; keeping only the last two cells of that array gives back the O(1)-space iterative method.

> **NOTE ON THE CODE**
>
> `fib_memo(n, memo={})` uses Python's **mutable default argument**: the same dictionary survives between calls, so later calls are even faster. That is convenient here, but in general it is a well-known Python pitfall — a viva examiner may ask about it.

Comparison

Method	Time	Space	Comment
Iterative	O(n)	O(1)	fastest and simplest; preferred in practice
Recursive	O(2ⁿ)	O(n)	elegant, but unusable beyond n ≈ 35
Memoized recursion	O(n)	O(n)	shows how DP fixes overlapping sub-problems
Matrix power / fast doubling	O(log n)	O(1) or O(log n)	uses [[1,1],[1,0]]ⁿ — advanced bonus

Probable viva questions — Practical 1

Q1. What is the Fibonacci series and its recurrence relation?

A series where each term is the sum of the previous two: 0, 1, 1, 2, 3, 5, 8, … Recurrence: F(n) = F(n−1) + F(n−2), with base cases F(0) = 0 and F(1) = 1.

Q2. What is recursion? What are its two essential parts?

A function that calls itself on a smaller input. It needs a **base case** that stops the recursion and a **recursive case** that moves the input towards the base case.

Q3. What happens if the base case is missing?

The function calls itself forever until the call stack overflows. In Python this raises `RecursionError: maximum recursion depth exceeded` (default limit about 1000).

Q4. Why is recursive Fibonacci so slow?

Because of overlapping sub-problems: fib(n) calls fib(n−1) and fib(n−2), and fib(n−1) calls fib(n−2) again, so the same values are recomputed exponentially many times.

Q5. Give the time and space complexity of each method.

Iterative: time O(n), space O(1). Recursive: time $O(2^n)$ (precisely $\Theta(\varphi^n)$), space O(n) for the stack. Memoized: time O(n), space O(n).

Q6. Why is the space of recursive Fibonacci only O(n) if it makes $O(2^n)$ calls?

Space counts calls that are alive **at the same time**. The calls run depth-first, so at any moment only one path from the root to a leaf is on the stack, and that path has at most n frames.

Q7. How many function calls does fib_recursive(5) make? What about fib(30)?

15 for fib(5). In general calls(n) = $2 \cdot F(n+1) - 1$, so fib(30) makes $2 \times 1{,}346{,}269 - 1 = 2{,}692{,}537$ calls, which matches the program's output.

Q8. What is memoization? How is it different from tabulation?

Memoization = top-down: recurse normally but cache each result the first time. Tabulation = bottom-up: fill a table from the smallest sub-problem upward with a loop. Both are dynamic programming and both make Fibonacci O(n).

Q9. Which properties make a problem suitable for dynamic programming?

Overlapping sub-problems (the same smaller problem is needed many times) and **optimal substructure** (the answer can be built from answers of sub-problems). Fibonacci has both.

Q10. Recursion vs iteration — which is better here and why?

Iteration: no function-call overhead and O(1) space. Recursion is shorter and closer to the mathematical definition but slow and stack-hungry. For Fibonacci, iteration is clearly better.

Q11. What is tail recursion? Is fib_recursive tail recursive?

A call is tail recursive when the recursive call is the very last operation, so a compiler can reuse the stack frame (turning it into a loop). fib_recursive is not — after both calls return it still has to add them. Python does not do tail-call optimisation anyway.

Q12. How does `a, b = b, a + b` work without a temporary variable?

Python evaluates the whole right-hand side first (packing it into a tuple) and then assigns, so a gets the old b and b gets old a + old b.

Q13. Can Fibonacci be computed in less than O(n) time?

Yes. Using the identity $[[1,1],[1,0]]^n = [[F(n+1),F(n)],[F(n),F(n-1)]]$ and fast exponentiation (repeated squaring), F(n) is found in O(log n) multiplications. Binet's formula $F(n) = (\varphi^n - \psi^n)/\sqrt{5}$ also exists but suffers from floating-point error for large n.

Q14. Why does the program use `time.perf_counter()`?

It is a high-resolution timer meant for measuring short durations; the difference of two readings gives the elapsed time of each method.

Q15. Why does the program reset `calls = 0` before each run?

`calls` is a global counter; without resetting it the count from the previous n would be added to the next one.

Q16. Why do Fibonacci numbers not overflow in Python?

Python integers have arbitrary precision; they grow as large as memory allows. In C/Java an `int` overflows around F(47) and a 64-bit `long` around F(93).

Q17. Where are Fibonacci numbers used in computing?

Fibonacci heaps (used in fast Dijkstra/Prim), Fibonacci search, analysis of Euclid's GCD algorithm (worst case is consecutive Fibonacci numbers), and as a standard example to teach recursion and DP.

PRACTICAL 2

Huffman encoding — Greedy strategy

Aim. Write a program to implement Huffman encoding using a greedy strategy.

Code: `DAA_HuffmanEncoding_pract2.py` — `Node` class, `build_tree` (min-heap), `make_codes`, `encode`, `decode`, and a driver that runs "mississippi", "aaaabbc" and "aaaa".

Why compress, and the idea of variable-length codes

Normal text uses a **fixed-length code**: every character takes 8 bits in ASCII. But in real text some characters appear far more often than others. Huffman's idea: **give frequent characters short codes and rare characters long codes**, so the total number of bits is smaller. It is **lossless** — decoding gives back exactly the original text.

Prefix codes

With variable lengths there is a danger of ambiguity. If a = 0 and b = 01, then "01" could be "b" or "a" followed by something. Huffman produces a **prefix code (prefix-free code)**: **no code is a prefix of another code**. Then the bit stream can be decoded left to right without any separators — the first code you complete is always the right one. Characters sit only at **leaves** of the binary tree, which is exactly why no code can be a prefix of another.

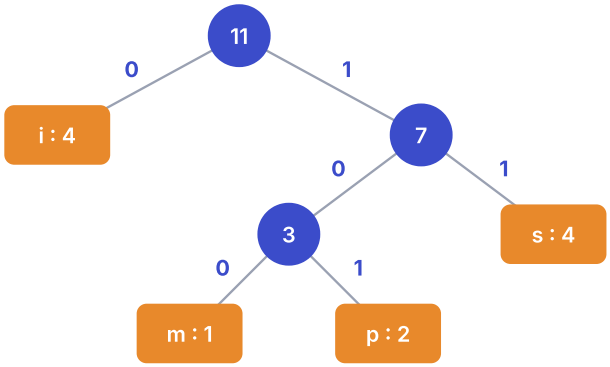
The greedy algorithm

1. Count the frequency of each character.
2. Make a leaf node for each character and put all nodes in a **min-priority queue (min-heap)** keyed on frequency.
3. **Greedy step:** remove the two nodes with the smallest frequencies, x and y.
4. Create a new internal node with frequency x + y, left child x, right child y, and insert it back.
5. Repeat steps 3–4 until one node remains — the root. (For n distinct characters there are exactly n − 1 merges.)
6. Walk the tree: left edge = 0, right edge = 1. The path from root to a leaf is that character's code.

Worked example — "mississippi"

Frequencies: **m = 1, p = 2, i = 4, s = 4** (11 characters).

Step	Heap before	Take two smallest	New node
1	m1, p2, i4, s4	m1 + p2	3
2	3, i4, s4	3 + s4	7
3	i4, 7	i4 + 7	11 (root)



Huffman tree for "mississippi". Circles are internal nodes (sum of children); characters are only at the leaves.

Char	Freq	Code	Bits used
i	4	0	4 × 1 = 4
s	4	11	4 × 2 = 8
m	1	100	1 × 3 = 3
p	2	101	2 × 3 = 6
Total	11		**21 bits**

Encoded: `100 0 11 11 0 11 11 0 101 101 0` = **10001111010111101011010**.

- 8-bit ASCII: 11 × 8 = **88 bits** → Huffman saves about 76%.
- Even the best **fixed-length** code for 4 symbols needs 2 bits each: 22 bits. Huffman's 21 is still better.
- Average code length = 21 / 11 ≈ **1.91 bits per character**. (The theoretical minimum, the Shannon entropy of this text, is about 1.82 bits.)

Decoding `100011…` : read 1 → not a code; 10 → not a code; 100 → **m**; then 0 → **i**; 11 → **s** … and so on. Because the code is prefix-free, the first match is always correct — this is exactly what the `decode()` function does with its `current` buffer.

Edge case — "aaaa". With only one distinct character the tree is a single leaf and the path is empty, so the code assigns it `"0"` (`codes[node.char] = code or "0"`). Without that line the encoded string would be empty.

Why greedy works here (proof idea)

1. **Greedy-choice property:** there is always an optimal tree in which the two least-frequent characters are siblings at the deepest level. Swapping a rarer character deeper never increases the cost.
2. **Optimal substructure:** after merging those two into one "super-character" of frequency x + y, an optimal tree for the smaller problem expands into an optimal tree for the original.

Together these prove Huffman always produces an **optimal prefix code** (minimum total bits Σ freq × code length).

Complexity

With n distinct characters: building the heap O(n), then n − 1 merges, each doing two pops and one push at O(log n) → **O(n log n)** time. Counting frequencies of a text of length m is O(m). Space **O(n)** for

the heap and the 2n − 1 tree nodes.

Code points worth knowing

- `heapq` is a **min-heap**; `heappop` always returns the smallest element.
- `Node.__lt__` compares by frequency — `heapq` needs this to compare two Node objects.
- `Counter(text)` counts character frequencies in one line.
- `make_codes` appends "0" when going left and "1" when going right.

Probable viva questions — Practical 2

Q1. What is Huffman coding?

A lossless data-compression algorithm that assigns variable-length binary codes to characters — shorter codes to more frequent characters — built with a greedy algorithm on a binary tree. It was designed by David Huffman in 1952.

Q2. What is a greedy algorithm? Why is Huffman greedy?

A greedy algorithm makes the locally best choice at each step and never reconsiders it. Huffman always merges the two lowest-frequency nodes — the cheapest merge right now — and never changes that decision later.

Q3. What is a prefix code and why is it needed?

A code in which no codeword is a prefix of another. It lets us decode a continuous bit stream without separators, because as soon as the bits read so far match a codeword, that match is the only possibility.

Q4. How does the Huffman tree guarantee prefix codes?

Characters are stored only at leaves. A code is the root-to-leaf path; one leaf's path can never be the start of another leaf's path, because a leaf has no children.

Q5. Which data structure is used and why?

A **min-priority queue (min-heap)**. We repeatedly need the two smallest frequencies; a heap gives each removal and insertion in O(log n) instead of scanning the list in O(n).

Q6. What is the time complexity of Huffman coding?

O(n log n) for n distinct characters: n − 1 merges, each with heap operations costing O(log n). Counting frequencies adds O(m) for text length m.

Q7. Is Huffman coding lossless or lossy?

Lossless — decoding gives back exactly the original data. The program checks this with `decode(bits, codes) == text`.

Q8. Is the Huffman code unique?

No. When frequencies tie, or if we swap the 0/1 labels of left and right children, we get different codes. All of them have the same minimum total length, so they are equally optimal.

Q9. For "mississippi", what are the codes and how many bits are saved?

i = 0, s = 11, m = 100, p = 101. Encoded length 21 bits versus 88 bits in 8-bit ASCII (and 22 bits with a fixed 2-bit code).

Q10. How is the encoded data decoded?

Start at the root; read bits one at a time — 0 go left, 1 go right; when a leaf is reached, output its character and go back to the root. The program does the same with a reverse lookup table of codes.

Q11. How many internal nodes does a Huffman tree with n characters have?

n − 1 internal nodes (one per merge), so 2n − 1 nodes in total. It is a **full binary tree** — every internal node has exactly two children.

Q12. What must be sent along with the compressed data?

The code table (or the tree, or the frequencies) — the receiver needs it to decode. For small inputs this overhead can be larger than the savings.

Q13. When does Huffman coding give no benefit?

When all characters have equal frequency (and their count is a power of two), the tree is perfectly balanced and the codes are as long as a fixed-length code.

Q14. What is the difference between fixed-length and variable-length coding?

Fixed-length gives every character the same number of bits (ASCII 8, or $\lceil \log_2 n \rceil$ for n symbols). Variable-length gives different lengths based on frequency, using fewer bits on average.

Q15. Where is Huffman coding used in real life?

Inside DEFLATE (ZIP, gzip, PNG), JPEG and MP3 compression (as the final entropy-coding stage), and in fax machines.

Q16. Name another greedy algorithm and say whether greedy always gives the optimal answer.

Fractional knapsack, Prim's and Kruskal's MST, Dijkstra's shortest path, activity selection. Greedy is not always optimal — for example it fails on 0/1 knapsack and on coin change with denominations {1, 3, 4} for amount 6 (greedy gives 4+1+1, optimal is 3+3).

Q17. What is the role of `__lt__` in the Node class?

`heapq` compares elements with `<`. Defining `__lt__` tells Python that a node is "smaller" when its frequency is smaller, so the heap orders nodes by frequency.

PRACTICAL 3

0/1 Knapsack — Dynamic Programming

Aim. Write a program to solve the 0/1 knapsack problem using dynamic programming.

Code: `DAA_Knapsack01_pract3.py` — `knapsack()` fills the table and traces back the chosen items; `solve()` prints the table. Two problems are solved: (w = [2,3,4,5], v = [3,4,5,6], W = 5) and (w = [3,4,6,5], v = [2,3,1,4], W = 8).

The problem

You have n items; item i has weight w_i and value v_i. A bag can carry at most W. Choose items to **maximise total value** without exceeding W. "**0/1**" means each item is either taken completely (1) or left (0) — no fractions, no repeats.

Trying every subset costs **O(2ⁿ)**. Greedy (by value/weight ratio) can give wrong answers. Dynamic programming gives the exact optimum in **O(n·W)**.

Building the DP solution

State. `dp[i][w]` = the best value achievable using only the **first i items** with capacity **w**.

Choice for item i.

- **Don't take it:** value stays `dp[i−1][w]`.
- **Take it** (only if $w_i \le w$): value = v_i + `dp[i−1][w − wᵢ]` — its value plus the best we can do with the remaining capacity using earlier items.

RECURRENCE

dp[i][w] = dp[i−1][w] if $w_i > w$

dp[i][w] = max(dp[i−1][w], v_i + dp[i−1][w − w_i]) otherwise

Base: dp[0][w] = 0 (no items) and dp[i][0] = 0 (no capacity). **Answer: dp[n][W].**

We use row i − 1 (not row i) for the "take" case. That is what makes it **0/1** — each item can be used at most once. Using row i would allow unlimited copies (the *unbounded* knapsack).

Worked example 1 — weights [2, 3, 4, 5], values [3, 4, 5, 6], W = 5

i (item: w, v)	w=0	1	2	3	4	5
0 (none)	0	0	0	0	0	0
1 (w2, v3)	0	0	**3**	3	3	3
2 (w3, v4)	0	0	3	4	4	**7**
3 (w4, v5)	0	0	3	4	5	**7**
4 (w5, v6)	0	0	3	4	5	**7**

How one cell is filled — dp[2][5] (item 2: w = 3, v = 4): don't take → dp[1][5] = 3; take → 4 + dp[1][5−3] = 4 + dp[1][2] = 4 + 3 = 7. max(3, 7) = **7**.

Trace back (the highlighted cells), starting at dp[4][5] = 7:

1. dp[4][5] = 7 = dp[3][5] → item 4 **not** taken.
2. dp[3][5] = 7 = dp[2][5] → item 3 **not** taken.
3. dp[2][5] = 7 ≠ dp[1][5] = 3 → item 2 **taken**; capacity 5 − 3 = 2.
4. dp[1][2] = 3 ≠ dp[0][2] = 0 → item 1 **taken**; capacity 2 − 2 = 0.

Answer: maximum value 7, items {1, 2}, total weight 5.

Worked example 2 — weights [3, 4, 6, 5], values [2, 3, 1, 4], W = 8

i	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (w3,v2)	0	0	0	2	2	2	2	2	2
2 (w4,v3)	0	0	0	2	3	3	3	5	5
3 (w6,v1)	0	0	0	2	3	3	3	5	5
4 (w5,v4)	0	0	0	2	3	4	4	5	**6**

Trace back: dp[4][8] = 6 ≠ 5 → item 4 taken (capacity 3); dp[3][3] = dp[2][3] → item 3 no; dp[2][3] = dp[1][3] → item 2 no; dp[1][3] = 2 ≠ 0 → item 1 taken. **Answer: value 6, items {1, 4}, weight 8.**

Why greedy fails for 0/1 knapsack

Items (w, v): A (10, 60), B (20, 100), C (30, 120), W = 50. Ratios: A = 6, B = 5, C = 4. Greedy by ratio takes A, then B (weight 30, value 160); C no longer fits. Optimal is **B + C = 220**. In the **fractional** knapsack greedy is optimal — after A and B we would take 20/30 of C for 80 more, total 240.

	0/1 Knapsack	Fractional Knapsack
Item can be split?	No	Yes
Method	Dynamic Programming	Greedy by v/w ratio
Time	O(n·W)	O(n log n) (sorting)
Greedy optimal?	No	Yes

Complexity

Time O(n·W) — (n+1)·(W+1) cells, each filled in O(1); traceback adds O(n). **Space O(n·W)** for the table. Because only the previous row is needed, it can be reduced to **O(W)** with a single 1-D array filled from **right to left** (so that dp[w − w_i] still holds the previous row's value).

O(n·W) looks polynomial but is **pseudo-polynomial**: W is a number, and its input size is only log W bits. If W is huge (say 10^{12}) the table is impossible to build. 0/1 knapsack is **NP-hard** in general.

Probable viva questions — Practical 3

Q1. State the 0/1 knapsack problem.

Given n items with weights and values and a bag of capacity W, choose a subset of items with total weight ≤ W that maximises total value; each item is either taken whole or not at all.

Q2. Why is it called 0/1?

Each item has only two choices: 0 (leave it) or 1 (take it completely). You cannot take a fraction or take it twice.

Q3. What is dynamic programming?

A technique that solves a problem by breaking it into overlapping sub-problems, solving each sub-problem once, storing its answer in a table, and building larger answers from stored ones.

Q4. What are the two properties needed for DP? Show them in knapsack.

Optimal substructure: the best solution for (i, w) is built from the best solution of (i−1, w) or (i−1, w−w_i). **Overlapping sub-problems:** many different branches need the same (i, w) value, so storing it saves time.

Q5. Write the recurrence relation.

dp[i][w] = max(dp[i−1][w], v_i + dp[i−1][w − w_i]) when $w_i \le w$, otherwise dp[i][w] = dp[i−1][w]; with dp[0][w] = dp[i][0] = 0.

Q6. What does dp[i][w] mean?

The maximum value obtainable using only the first i items with a bag of capacity w.

Q7. How do you find which items were selected?

Trace back from dp[n][W]. If dp[i][w] ≠ dp[i−1][w], item i was taken — record it and reduce w by its weight. Otherwise skip it. Move up to row i − 1 and repeat.

Q8. What is the time and space complexity?

Time O(n·W) and space O(n·W). Space can be reduced to O(W) with a one-dimensional array, but then the traceback is no longer possible without extra storage.

Q9. Why is O(n·W) called pseudo-polynomial?

It is polynomial in the **value** of W but exponential in the **number of bits** needed to write W. Doubling the number of digits of W multiplies the work enormously.

Q10. Is 0/1 knapsack NP-complete?

Its decision version ("is there a subset with value ≥ V and weight ≤ W?") is NP-complete. The DP does not contradict this because it is pseudo-polynomial.

Q11. Why does greedy fail for 0/1 knapsack? Give an example.

Greedy picks the best ratio first and cannot undo it. Items (10, 60), (20, 100), (30, 120) with W = 50: greedy gets 160, the optimum (items 2 and 3) is 220.

Q12. Difference between 0/1 knapsack and fractional knapsack?

Fractional allows taking part of an item and is solved optimally by greedy in O(n log n). 0/1 does not allow fractions and needs DP (or branch and bound / backtracking).

Q13. What is the answer for weights [2, 3, 4, 5], values [3, 4, 5, 6], W = 5?

Maximum value 7 by taking items 1 and 2 (weights 2 + 3 = 5).

Q14. Why does the "take" case use row i − 1 and not row i?

Row i − 1 contains solutions that do not use item i yet. Using row i would let item i be added again, giving the unbounded knapsack.

Q15. If you use a 1-D array, why must the inner loop run from W down to w_i?

Going right to left ensures dp[w − w_i] is still the old (previous-item) value when it is read. Going left to right would reuse the current item multiple times.

Q16. Name other methods to solve 0/1 knapsack.

Brute force (2^n subsets), recursion with memoization (top-down DP), backtracking, and branch and bound (uses an upper bound from the fractional knapsack to prune).

Q17. Real-life applications of knapsack?

Choosing investments within a budget, loading cargo, selecting tasks for limited CPU time or memory, and cutting raw material to reduce waste.

PRACTICAL 4

N-Queens with the first queen placed — Backtracking

Aim. Design an n-Queens matrix with the first queen already placed. Use backtracking to place the remaining queens and print the final matrix.

Code: `DAA_NQueens_pract4.py` — `is_safe`, recursive `place` (skips the fixed row), `n_queens`, and `show` which prints a 0/1 matrix. Tested on 4-Queens with the first queen at (1, 2) and (1, 1), and 8-Queens with the first queen at (1, 1).

The problem

Place N queens on an N × N chessboard so that **no two queens attack each other** — no two in the same **row**, **column** or **diagonal**. Here the first queen's position is given by the user and must not move.

Representation

The code stores the board as a 1-D list: `board[r] = c` means the queen in row r is in column c (−1 = empty). This automatically guarantees one queen per row, so only columns and diagonals need checking.

Diagonal test: two queens at (r_1, c_1) and (r_2, c_2) are on the same diagonal when $|r_1 - r_2| = |c_1 - c_2|$ — the row distance equals the column distance.

Backtracking

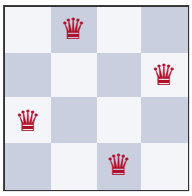
Backtracking builds the solution one row at a time. In each row it tries columns from left to right. If a column is safe, it places the queen and moves to the next row. If no column in a row works, it **backtracks**: returns to the previous row, removes that queen (`board[row] = -1`) and tries that row's next column. This is a **depth-first search on the state-space tree**, and `is_safe` is the **bounding function** that prunes a branch as soon as it violates the rules.

```
place(row):
    if row == N: return True                 # all rows filled -> solution
    if row == fixed_row: return place(row+1) # the given queen stays where it is
    for col in 0..N-1:
        if is_safe(row, col):
            board[row] = col
            if place(row+1): return True
            board[row] = -1                  # BACKTRACK
    return False
```

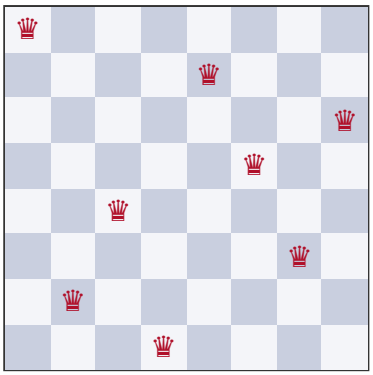
Worked example — 4-Queens, first queen at row 1, column 2

Board (0-indexed): `[1, -1, -1, -1]`.

1. Row 0 is fixed → go to row 1.
2. Row 1: col 0 (diagonal with (0,1)) ×, col 1 (same column) ×, col 2 (diagonal) ×, **col 3** ✓ → `[1, 3, -1, -1]`.
3. Row 2: **col 0** ✓ (not same column as 1 or 3, not diagonal) → `[1, 3, 0, -1]`.
4. Row 3: col 0 ×, col 1 ×, **col 2** ✓ → `[1, 3, 0, 2]`. All rows filled — done.



4-Queens, first queen at (1, 2) → matrix 0100 / 0001 / 1000 / 0010



8-Queens, first queen at (1, 1) → columns 1, 5, 8, 6, 3, 7, 2, 4

First queen at the corner (1, 1) for N = 4: no solution. 4-Queens has only two solutions, [1, 3, 0, 2] and [2, 0, 3, 1], and neither has a queen in a corner, so the program correctly prints "No solution possible with this first queen" after exhausting all branches.

Facts to remember

N	1	2	3	4	5	6	7	8
Number of solutions	1	0	0	2	10	4	40	**92**

8-Queens has 92 solutions, of which **12 are fundamentally distinct** (the rest are rotations and reflections).

Complexity

The first row has N choices, the next at most N − 1 (one column used), and so on → worst case **O(N!)**; with the O(N) `is_safe` check it is O(N · N!). Pruning makes the real number of steps far smaller. **Space O(N)** — the board list plus a recursion depth of N.

Faster safety check: keep three sets — used columns, used "r − c" values (one diagonal direction) and used "r + c" values (the other) — and check in O(1).

Probable viva questions — Practical 4

Q1. What is the N-Queens problem?

Placing N queens on an N × N board so that no two queens share a row, column or diagonal.

Q2. What is backtracking?

A systematic trial-and-error method: build the solution step by step; when a partial solution breaks a constraint, undo the last step (backtrack) and try the next option. It is depth-first search with pruning.

Q3. What is a state-space tree?

A tree of all possible partial solutions: the root is the empty board, level k has one queen in each of the first k rows. Backtracking explores it depth-first and cuts off branches that cannot lead to a solution.

Q4. What is a bounding function in backtracking?

A test that decides whether a partial solution can still lead to a valid one. Here it is `is_safe` — if it fails, the whole sub-tree below is skipped.

Q5. How do you check if two queens are on the same diagonal?

If $|r_1 - r_2| = |c_1 - c_2|$. Equivalently, queens on one diagonal have the same r − c, and on the other diagonal the same r + c.

Q6. Why don't we check rows in `is_safe`?

We place exactly one queen per row (board[r] holds one column), so two queens can never be in the same row.

Q7. What happens when the first queen is placed at a position with no solution?

The algorithm tries every column in every remaining row, backtracks all the way, and `place` returns False. The program then prints that no solution is possible.

Q8. How many solutions does 8-Queens have? 4-Queens?

8-Queens: 92 (12 distinct up to symmetry). 4-Queens: 2.

Q9. For which values of N is there no solution?

N = 2 and N = 3. Every N ≥ 4 (and N = 1) has at least one solution.

Q10. What is the time complexity of N-Queens backtracking?

Worst case O(N!) placements (O(N · N!) with the O(N) safety check). In practice pruning makes it much faster. Space O(N).

Q11. Difference between backtracking and brute force?

Brute force generates every complete arrangement and checks each one at the end (for 8-Queens, C(64, 8) ≈ 4.4 billion placements). Backtracking checks constraints after each step and abandons bad partial solutions early.

Q12. Difference between backtracking and branch and bound?

Backtracking uses DFS and finds feasible solutions (decision problems like N-Queens). Branch and bound is for optimisation problems: it uses bounds on the best possible value to prune, and may explore with BFS or best-first search (e.g. 0/1 knapsack, TSP).

Q13. Why does the `place` function skip `fixed_row`?

That row already holds the user's first queen, which must not move. The function simply moves on to the next row.

Q14. What does `board[row] = -1` do?

It removes the queen from that row — this is the actual "backtrack" step, undoing the choice before trying the next column.

Q15. How can the safety check be made O(1)?

Maintain three boolean arrays or sets: columns used, r − c diagonals used and r + c anti-diagonals used. A square is safe if its column, r − c and r + c are all unused.

Q16. Name other problems solved by backtracking.

Sudoku, graph colouring, Hamiltonian cycle, sum of subsets, maze / rat-in-a-maze, and generating permutations or combinations.

PRACTICAL 5

Quick Sort — deterministic and randomized

Aim. Write a program to analyse Quick Sort, showing the analysis of both the deterministic and the randomized variant.

Code: `DAA_QuickSort_pract5.py` — Lomuto `partition`, one `quick_sort` with a `randomized` flag, a global `comparisons` counter, and a test table on random, sorted and reverse-sorted inputs of size 1000.

How Quick Sort works (divide and conquer)

1. **Divide:** choose a **pivot** and **partition** the array so that everything ≤ pivot is on its left and everything > pivot is on its right. The pivot is now in its final sorted position.
2. **Conquer:** recursively sort the left part and the right part.
3. **Combine:** nothing to do — the array is already sorted in place.

Lomuto partition (used in the code)

Pivot = last element. Index `i` marks the end of the "≤ pivot" region. Scan `j` from low to high − 1; whenever `arr[j] ≤ pivot`, grow the region (`i += 1`) and swap `arr[j]` into it. Finally swap the pivot into position i + 1.

Trace on [10, 7, 8, 9, 1, 5], pivot = 5:

j	arr[j]	≤ 5?	action	array
0	10	no	—	[10, 7, 8, 9, 1, 5]
1	7	no	—	[10, 7, 8, 9, 1, 5]
2	8	no	—	[10, 7, 8, 9, 1, 5]
3	9	no	—	[10, 7, 8, 9, 1, 5]
4	1	yes	i = 0, swap arr[0] ↔ arr[4]	[1, 7, 8, 9, 10, 5]
end			swap pivot into i + 1 = 1	[1, **5**, 8, 9, 10, 7]

Now 5 is in its final place. Recursing on [8, 9, 10, 7] (pivot 7) → [**7**, 9, 10, 8], then [9, 10, 8] (pivot 8) → [**8**, 10, 9], then [10, 9] (pivot 9) → [**9**, 10]. Result: **[1, 5, 7, 8, 9, 10]**.

Deterministic vs randomized

Deterministic (always last element as pivot): if the input is already sorted or reverse-sorted, the pivot is always the largest or smallest element. Each partition then removes just one element, giving sub-problems of size n − 1 and 0.

Randomized: pick a random index, swap it to the end, then partition as usual. No particular input can force bad splits consistently; the *expected* running time is O(n log n) for **every** input. The randomness is in the algorithm, not in the data.

Analysis

Case	When	Recurrence	Time
Best	pivot always splits into two equal halves	$T(n) = 2T(n/2) + n$	**$O(n \log n)$**
Average	random input (or random pivot)	splits are "good enough" on average	**$O(n \log n)$** $\approx 1.39\, n \log_2 n$ comparisons
Worst	pivot always smallest/largest (sorted input, deterministic)	$T(n) = T(n-1) + n$	**$O(n^2)$** $= n(n-1)/2$ comparisons

Worst-case expansion: $T(n) = n + (n-1) + (n-2) + \ldots + 1 = n(n+1)/2 \rightarrow O(n^2)$.

What the program shows for n = 1000:

- Sorted or reverse-sorted input, deterministic → exactly **1000 × 999 / 2 = 499,500 comparisons** (worst case). The recursion is 1000 levels deep, which is why the code raises Python's recursion limit with `sys.setrecursionlimit(10000)`.
- Randomized on the same input → roughly **10,000–12,000 comparisons** (the expected value for random pivots is $2(n+1)H_n - 4n \approx 11{,}000$ for n = 1000).
- Random input → both variants behave like the average case.

Space: O(log n) recursion stack on average, O(n) in the worst case. Quick Sort is **in-place** (no extra array) but **not stable** (equal elements can change relative order).

Quick Sort vs Merge Sort

	Quick Sort	Merge Sort
Worst case	$O(n^2)$	O(n log n)
Average	O(n log n), small constant — usually faster in practice	O(n log n)
Extra space	O(log n) stack, in-place	O(n) extra array
Stable?	No	Yes
Work done in	partition (before recursion)	merge (after recursion)

Probable viva questions — Practical 5

Q1. Explain Quick Sort.

A divide-and-conquer sorting algorithm: choose a pivot, partition the array so smaller elements go left and larger go right (the pivot lands in its final position), then recursively sort both sides. No merge step is needed.

Q2. What is a pivot? How can it be chosen?

The element around which the array is partitioned. Choices: first element, last element (this code), middle element, random element (randomized Quick Sort), or median-of-three (median of first, middle and last).

Q3. What is partitioning? Name the two common schemes.

Rearranging the sub-array so all elements ≤ pivot come before it and the rest after it. **Lomuto** (pivot = last element, one index scanning — used here) and **Hoare** (two indices moving towards each other; fewer swaps on average).

Q4. What are the best, average and worst-case complexities?

Best O(n log n), average O(n log n), worst $O(n^2)$. Space O(log n) average, O(n) worst (recursion stack).

Q5. When does the worst case occur for deterministic Quick Sort?

When the pivot is always the smallest or largest element — with last-element pivot that happens for already sorted or reverse-sorted input (and for arrays where all elements are equal in Lomuto's scheme).

Q6. Derive the worst-case complexity.

$T(n) = T(n-1) + (n-1)$ comparisons → $(n-1) + (n-2) + \ldots + 1 = n(n-1)/2 \rightarrow O(n^2)$. For n = 1000 that is 499,500, exactly what the program reports.

Q7. Derive the best-case complexity.

$T(n) = 2T(n/2) + n$. The recursion tree has $\log_2 n$ levels and each level does about n work → O(n log n). By the Master theorem, a = 2, b = 2, f(n) = n = n^($\log_2 2$) → case 2 → Θ(n log n).

Q8. What is Randomized Quick Sort and why is it better?

It picks the pivot at random. No fixed input (like a sorted array) can make it slow every time, so its expected time is O(n log n) for all inputs. The worst case is still $O(n^2)$ but its probability is extremely small.

Q9. Is Randomized Quick Sort a Las Vegas or Monte Carlo algorithm?

Las Vegas — the output is always correct; only the running time is random. A Monte Carlo algorithm has fixed running time but may give a wrong answer with small probability (e.g. Miller–Rabin primality test).

Q10. Is Quick Sort stable? Is it in-place?

Not stable — swaps can change the relative order of equal keys. It is in-place — only O(log n) stack space, no auxiliary array.

Q11. Why is Quick Sort usually faster than Merge Sort in practice despite its $O(n^2)$ worst case?

Smaller constant factors, in-place operation, and good cache behaviour because it scans memory sequentially. With a random pivot the worst case practically never happens.

Q12. Why does the program call `sys.setrecursionlimit(10000)`?

For sorted input, the deterministic version recurses about n = 1000 levels deep, which exceeds Python's default limit of about 1000 and would crash with RecursionError.

Q13. Why does the code count comparisons instead of only measuring time?

Comparisons are machine-independent and match the theoretical analysis exactly (e.g. 499,500 in the worst case). Time depends on the computer and other running programs.

Q14. What does `random.seed(1)` do?

It fixes the starting point of the random number generator, so the random input and random pivots are the same every run and the output is repeatable for the write-up.

Q15. How can the worst case be avoided apart from random pivots?

Median-of-three pivot selection; switching to Heap Sort when the recursion gets too deep (**Introsort**, used by C++ `std::sort`); three-way partitioning for many duplicate keys.

Q16. What is three-way partitioning?

Dividing into three parts: less than, equal to, and greater than the pivot (Dutch National Flag). Equal elements are not recursed on again, so arrays with many duplicates sort in near-linear time.

Q17. What is the lower bound for comparison-based sorting?

$\Omega(n \log n)$. A decision tree that distinguishes all n! orderings needs height at least $\log_2(n!) \approx n \log_2 n$.

REVISION

Quick revision and cross-practical questions

One-page summary

Practical	Strategy	Key idea	Time	Space
1 Fibonacci – iterative	Iteration	keep last two values	O(n)	O(1)
1 Fibonacci – recursive	Recursion	F(n−1) + F(n−2), recomputes	$O(2^n)$	O(n)
1 Fibonacci – memoized	DP (top-down)	cache each F(i)	O(n)	O(n)
2 Huffman	Greedy	merge two smallest frequencies	O(n log n)	O(n)
3 0/1 Knapsack	DP (bottom-up)	max(skip, take) table	O(n·W)	O(n·W) → O(W)
4 N-Queens	Backtracking	place row by row, undo on conflict	O(N!)	O(N)
5 Quick Sort – deterministic	Divide & conquer	last-element pivot	avg O(n log n), worst $O(n^2)$	O(log n) avg
5 Quick Sort – randomized	Randomized D&C	random pivot	expected O(n log n)	O(log n) avg

Mixed questions examiners like

Q1. Which of your practicals use dynamic programming?

Practical 3 (0/1 knapsack, bottom-up table) and the memoized version in Practical 1 (top-down DP for Fibonacci).

Q2. Which practical uses a greedy approach and which a backtracking approach?

Greedy: Huffman encoding (Practical 2). Backtracking: N-Queens (Practical 4).

Q3. Which of your practicals is divide and conquer? How is it different from DP?

Quick Sort (Practical 5). Its sub-problems (left and right parts) are independent and never overlap, so there is nothing to store. DP problems like knapsack have overlapping sub-problems.

Q4. Which practical shows the difference between exponential and polynomial time most clearly?

Practical 1: recursive Fibonacci takes $O(2^n)$ (2.7 million calls for n = 30) while the iterative version takes O(n) (30 steps).

Q5. In which practical does the input order change the running time?

Quick Sort — deterministic Quick Sort is $O(n^2)$ on sorted input but O(n log n) on random input. Randomized Quick Sort removes this dependence.

Q6. Which practicals use recursion? What is the maximum recursion depth in each?

Recursive/memoized Fibonacci (depth n), Huffman `make_codes` (depth = tree height ≤ n − 1), N-Queens `place` (depth N), Quick Sort (depth log n on average, n in the worst case).

Q7. Which data structures did you use?

Python lists (DP table, board, arrays), dictionary (memo, Huffman codes), min-heap via `heapq` (Huffman), a binary tree of Node objects (Huffman), and `Counter` for frequencies.

Q8. What is an optimisation problem vs a decision problem? Classify your practicals.

Optimisation: find the best value — knapsack (maximise value), Huffman (minimise total bits). Decision / search: find any valid arrangement — N-Queens. Sorting and Fibonacci are computation problems.